

Lab Session - RL

2026 삼성전자 DS 14기

2026. 07. 02.

실습 전 준비사항

실습 코드 변경사항 업데이트

git 명령어 사용해 프로젝트 폴더 다운로드받았다면

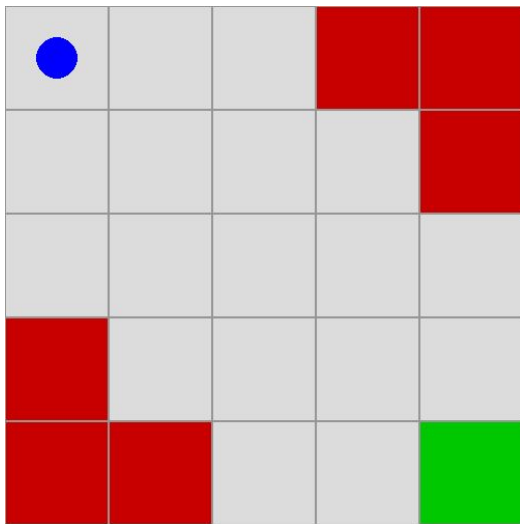
- 실습 프로젝트 폴더로 이동
- `git stash | git pull | git stash pop`

github 사이트에서 프로젝트 폴더 다운로드받았다면

- 다시 다운로드 https://github.com/SNU-IntelligentMotionLab/2026Spring_RL_Lab.git
- 가상환경 생성부터 반복
 - `python -m venv lab_env` 혹은 `py -m venv lab_env`로 명령어 입력.
 - `.\lab_env\Scripts\activate`
 - `pip install -r requirements.txt` 입력

이후 **Lab4_PG** 디렉토리로 이동 (`cd Lab4_PG`)

Continuous-Action GridWorld



`gridworld_c2.py` represents the agent state as a continuous (`row`, `col`) position. The action is a 2D vector in $[-1, 1]^2$, which is clamped internally by the environment.

Rewards:

- Move: -1
- Trap: -100, episode ends
- Goal: +100, episode ends

Each map cell is one of:

- Normal 0
- Wall 1
- Trap 2
- Goal 3

REINFORCE

Training

Run `train.py` with REINFORCE.

```
python train.py --algo reinforce --map {MAP_NAME} --save_name {SAVE_NAME}
```

Arguments:

- `--algo`: reinforce
- `--map`: map0, map1, map2, or map3
- `--save_name`: checkpoint suffix. Default is `example`.
- `--episodes`: number of training episodes. Default is 1000.
- `--max-steps`: maximum steps per episode. Default is 100.
- `--render`: render the environment during training.
- `--logdir`: TensorBoard log directory. Default is `runs`.
- `--seed`: random seed. Default is 42.

Example:

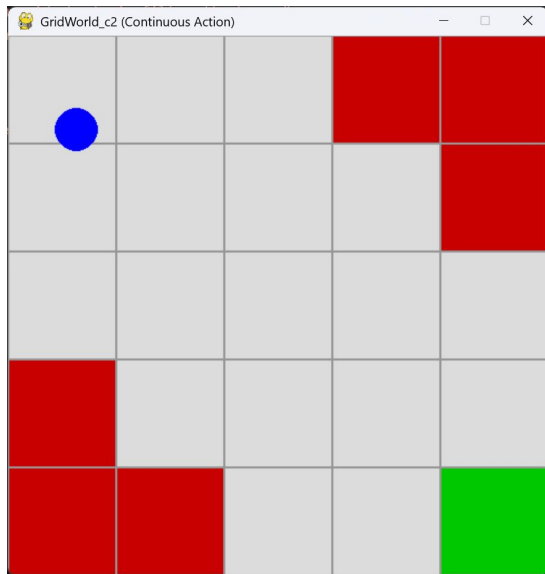
```
python train.py --algo reinforce --map map1 --save_name example --episodes 4000 --max-steps 50
```

REINFORCE

Testing

Run test.py with REINFORCE.

```
python test.py --algo reinforce --map {MAP_NAME} --save_name {SAVE_NAME}
```



REINFORCE

해당 주소로 접속

```
TensorBoard 2.20.0 at http://localhost:6006/ (Press CTRL+C to quit)
```

Visualizing REINFORCE

- use tensorboard

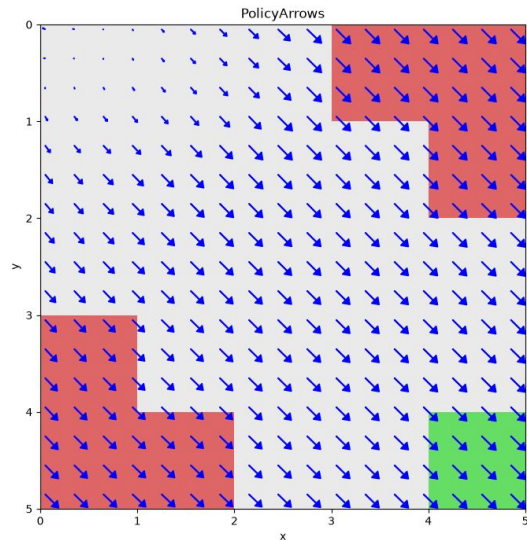
To Run TensorBoard

Run the following script to visualize TensorBoard logs.

```
tensorboard --logdir runs/
```

TensorBoard logs include:

- Loss
- PolicyArrows (figure visualization)



REINFORCE Walkthrough

Policy Gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi(s, a; \theta) q_{\pi}(s, a)]$$

- Update parameters by stochastic gradient ascent
- Using policy gradient theorem
- Using return G_t as an unbiased sample of $q_{\pi}(s_t, a_t)$

REINFORCE

Initialize θ arbitrarily

Repeat:

Sample an episode $\{s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T\} \sim \pi(s, a; \theta)$

For $t = 1$ to $T - 1$:

Compute return G_t

$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(s_t, a_t; \theta) G_t$

Until fixed iterations

Return θ

```
# reinforce.py:153
def train(self):
    map_img, H, W = log_map(self.writer, self.env)
    best_reward = float('-inf')

    for ep in range(1, self.args.episodes+1):
        state = self.env.reset()
        self.agent.reset_episode()
        total_R = 0.0

        if hasattr(self.agent, 'epsilon'):
            self.writer.add_scalar('Epsilon',
self.agent.epsilon, ep)

        # episode rollout
        for t in range(self.args.max_steps):
            action = self.agent.select_action(state)
            next_s, r, done, _ = self.env.step(action)
            total_R += r
            self.agent.rewards.append(r)
            state = next_s
            if self.args.render: self.env.render(tick=5000)
            if done: break

        # finish episode and update policy
        loss = self.agent.finish_episode()
        self.writer.add_scalar('Loss', loss, ep)
        self.writer.add_scalar('Reward', total_R, ep)
```

REINFORCE Walkthrough

REINFORCE

Initialize θ arbitrarily

Repeat:

Sample an episode $\{s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T\} \sim \pi(s, a; \theta)$

For $t = 1$ to $T - 1$:

Compute return G_t

$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(s_t, a_t; \theta) G_t$

Until fixed iterations

Return θ

```
def finish_episode(self):  
    # compute discounted returns  
    returns = []  
    G = 0  
    for r in reversed(self.rewards):  
        G = r + self.gamma * G  
        returns.insert(0, G)  
    returns = torch.tensor(returns, dtype=torch.float32,  
                           device=self.device)  
  
    if len(returns) > 1 and returns.std() > 1e-3:  
        returns = (returns - returns.mean()) /  
        (returns.std(unbiased=False) + 1e-9)
```


REINFORCE Walkthrough

REINFORCE

Initialize θ arbitrarily

Repeat:

Sample an episode $\{s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T\} \sim \pi(s, a; \theta)$

For $t = 1$ to $T - 1$:

Compute return G_t

$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(s_t, a_t; \theta) G_t$

Until fixed iterations

Return θ

```
dist = Normal(mean, std)
action = dist.sample()
action = torch.clamp(action, -1.0, 1.0)

log_prob = dist.log_prob(action).sum(dim=-1)
entropy = dist.entropy().sum(dim=-1)
```

Entropy loss (orange box)

Loss is high when entropy is low (negative sign)

Encourages high entropy → Encourages exploration

```
def finish_episode(self):
    # compute discounted returns
    returns = []
    G = 0
    for r in reversed(self.rewards):
        G = r + self.gamma * G
        returns.insert(0, G)
    returns = torch.tensor(returns, dtype=torch.float32,
                           device=self.device)

    if len(returns) > 1 and returns.std() > 1e-3:
        returns = (returns - returns.mean()) /
        (returns.std(unbiased=False) + 1e-9)

    # policy loss
    policy_losses = []
    for log_prob, Gt in zip(self.log_probs, returns):
        policy_losses.append(-log_prob * Gt)

    # entropy bonus
    entropy_bonus = torch.stack(self.entropies).sum()
    entropy_loss = -self.entropy_coeff * entropy_bonus

    loss = torch.stack(policy_losses).sum() + entropy_loss

    # update policy with gradient clipping
    self.optimizer.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(self.policy.parameters(),
                             self.max_grad_norm)
    self.optimizer.step()

    return loss.item()
```

Challenge: Implement Q actor-critic

DeepSARSA + REINFORCE

- Simple actor-critic algorithm based on action-value critic
- Using linear value fn. approximator $Q(s, a; w) = \phi(s, a)^T w$
 - Critic: Updates w by linear TD(0)
 - Actor: Updates θ by policy gradient

QAC

```
Initialize  $s, \theta, w$ 
Sample  $a \sim \pi(s, a; \theta)$ 
Repeat:
    Perform action  $a$ 
    Receive reward  $r$  and transition to  $s'$ 
    Sample  $a' \sim \pi(s', a'; \theta)$ 
     $\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(s, a; \theta) Q(s, a; w)$ 
     $w \leftarrow w + \beta (r + Q(s', a'; w) - Q(s, a; w)) \phi(s, a)$ 
     $a \leftarrow a', s \leftarrow s'$ 
Until fixed iterations
Return  $\theta$ 
```

Notes:

- DeepSARSA와 달리 continuous action이므로 Qnet 구조 변경이 필요
- policy loss에 Q값을 쓸 때 **.detach()** 하여 Qnet이 policy loss에 의해 학습되지 않도록 하기

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi(s_t, a_t; \theta) G_t]$$

REINFORCE

$$\approx \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi(s, a; \theta) Q(s, a; w)]$$

Q Actor-Critic

QnA